- 🛡️ TrustLens AI
  - ✨ Features
  - 🛠️ Tech Stack
  - 📁 Project Structure
  - 🚀 Setup & Installation
    - Prerequisites
    - 1. Backend Setup
    - 2. Frontend Setup
    - 3. Access the App
  - 🔌 API Endpoints
  - 🤖 AI Pipeline
    - Detection → Severity → Price → Explanation
    - YOLOv8 Model
    - Severity Scoring (Area-Weighted)
    - Price Recommendation
  - ⚙️ Environment Variables
  - 🚢 Deployment
    - Backend
    - Frontend
    - PostgreSQL (Production)

🛡️ TrustLens AI

AI-Powered Product Condition Verification Platform

Upload product images and receive instant damage detection, severity scoring, price recommendations, and AI-generated explanations — powered by a custom-trained YOLOv8 model.

✨ Features

- 🔍 **YOLOv8 Damage Detection** — Custom-trained model detecting **cracks, scratches, and stains**

- 📊 **Area-Weighted Severity Scoring** — Scores 0–10 using damage area % and class-specific weights
- 💰 **Price Recommendation** — Rule-based pricing engine with 5 discount tiers
- 📝 **AI Explanation** — Class-specific damage descriptions with resale recommendations
- 🔐 **JWT Authentication** — Secure signup, login, and protected routes
- 📁 **Report History** — All analyses saved and accessible from dashboard
- 🖼️ **Bounding Box Visualization** — Canvas-rendered detection overlays on images

---

🛠️ Tech Stack

Layer	Technology
Frontend	React 18 + Vite + Tailwind CSS
Backend	FastAPI + SQLAlchemy ORM
ML Model	YOLOv8n (Ultralytics) — 4 classes
Database	SQLite (dev) / PostgreSQL (prod)
Auth	JWT (access + refresh tokens) + bcrypt
Storage	Local filesystem (pluggable to S3/Supabase)

📁 Project Structure

```
TrustLensAI/
├── backend/
│   ├── main.py                 # FastAPI entry point
│   ├── config.py               # Environment config
│   ├── database.py             # SQLAlchemy setup
│   ├── .env                    # Environment variables
│   ├── requirements.txt
│   ├── model/
│   │   └── best_v2.pt          # Trained YOLOv8 weights
│   ├── models/                 # SQLAlchemy models
│   │   ├── user.py
│   │   ├── report.py
│   │   └── image.py
│   ├── schemas/                # Pydantic schemas
```

```
│   │   ├── auth.py
│   │   └── report.py
│   ├── routes/                 # API endpoints
│   │   ├── auth.py             # signup, login, logout, me
│   │   ├── upload.py           # image upload (1-5)
│   │   ├── analyze.py          # full AI pipeline
│   │   └── reports.py          # report history & detail
│   ├── services/               # Business logic
│   │   ├── auth.py             # JWT + bcrypt
│   │   ├── yolo.py             # YOLOv8 inference
│   │   ├── severity.py         # Area-weighted scoring
│   │   ├── pricing.py          # Price recommendation
│   │   ├── llm.py              # Explanation generator
│   │   └── storage.py          # File upload handler
│   └── uploads/                # Uploaded images
│
├── frontend/
│   ├── index.html
│   ├── package.json
│   ├── vite.config.js
│   ├── tailwind.config.js
│   └── src/
│       ├── main.jsx
│       ├── App.jsx
│       ├── index.css
│       ├── api/axios.js        # Axios + JWT interceptors
│       ├── context/AuthContext.jsx
│       ├── components/
│       │   ├── Navbar.jsx
│       │   ├── ProtectedRoute.jsx
│       │   └── LoadingSpinner.jsx
│       └── pages/
│           ├── Landing.jsx
│           ├── Signup.jsx
│           ├── Login.jsx
│           ├── Dashboard.jsx
│           ├── Upload.jsx
│           └── AnalysisResult.jsx
│
├── ML models/
│   ├── YOLOv8_Model/           # Training notebook
│   ├── merged_dataset/         # Dataset config
│   ├── merge.py                # Dataset merge script
│   └── requirements.txt
│
└── README.md
```

🚀 Setup & Installation

Prerequisites

- **Python 3.10+**
- **Node.js 18+** and npm

1. Backend Setup

```
cd backend

# Create & activate virtual environment
python -m venv venv
venv\Scripts\activate            # Windows
# source venv/bin/activate       # macOS/Linux

# Install dependencies
pip install -r requirements.txt
pip install ultralytics          # For YOLOv8

# Place your trained model
# Copy best_v2.pt to backend/model/best_v2.pt

# Start the server
uvicorn main:app --reload --port 8000
```

> **Note:** Without `best_v2.pt`, the app runs in **placeholder mode** with mock detections.

2. Frontend Setup

```
cd frontend
npm install
npm run dev
```

3. Access the App

Service	URL
Frontend	http://localhost:5173
Backend API	http://localhost:8000
Swagger Docs	http://localhost:8000/docs

🔌 API Endpoints

Method	Endpoint	Auth	Description
POST	`/auth/signup`	❌	Register new user
POST	`/auth/login`	❌	Login and get JWT
POST	`/auth/logout`	❌	Logout (stateless)
GET	`/auth/me`	✅	Current user profile
POST	`/upload`	✅	Upload 1–5 product images
POST	`/analyze`	✅	Run full AI analysis
GET	`/reports`	✅	All reports for user
GET	`/report/{id}`	✅	Single report detail

🤖 AI Pipeline

Detection → Severity → Price → Explanation

```
Image Upload → YOLOv8 Detection → Severity Scoring → Price Recommendation →
Explanation
```

YOLOv8 Model

- **Architecture:** YOLOv8n (nano) — fast inference
- **Classes:** `crack` (0), `good` (1), `scratch` (2), `stain` (3)
- **Training:** 50 epochs, 640×640, batch 16
- **Dataset:** Merged from per-class datasets (crack, scratch, stain, good)

Severity Scoring (Area-Weighted)

```
score = √(total_weighted_score / 10) × 3.16
```

Class	Weight	Impact
Crack	1.0	Highest — structural damage
Scratch	0.6	Medium — cosmetic
Stain	0.4	Lowest — cleanable

Price Recommendation

Severity	Condition	Discount
0	Good	0%
1–2	Minor	10%
3–4	Moderate	25%
5–7	Severe	40%
8–10	Critical	55%

⚙️ Environment Variables

Variable	Default	Description
`DATABASE_URL`	`sqlite:///./trustlens.db`	DB connection URL
`SECRET_KEY`	(set in .env)	JWT signing secret
`ALGORITHM`	`HS256`	JWT algorithm
`ACCESS_TOKEN_EXPIRE_MINUTES`	`30`	Token TTL
`OPENAI_API_KEY`	(empty)	For future LLM integration
`BACKEND_URL`	`http://localhost:8000`	Backend URL

Variable	Default	Description
`FRONTEND_URL`	`http://localhost:5173`	CORS origin

🚢 Deployment

Backend

```
pip install gunicorn
gunicorn main:app -w 4 -k uvicorn.workers.UvicornWorker --bind 0.0.0.0:8000
```

Frontend

```
npm run build
# Serve dist/ with Nginx, Vercel, or Netlify
```

PostgreSQL (Production)

```
DATABASE_URL=postgresql://user:password@host:5432/trustlens
```